



浙江财经大学

本科生毕业论文（设计）

题目：基于关键点检测的增强现实交互系统设计实现

学生姓名 徐山钧

学 号 190110910531

指导教师 陈琰宏

所在学院 信息管理与人工智能学院

专业名称 计算机科学与技术

班 级 19 计算机 1 班

2023 年 05 月

声明及论文使用的授权

本人郑重声明所呈交的论文是我个人在导师的指导下独立完成的。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写的研究成果。

论文作者签名: 年 月 日

本人同意浙江财经大学有关保留使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以上网公布全部内容，可以采用影印、缩印或其他复制手段保存论文。

论文作者签名： 年 月 日

基于关键点检测的增强现实交互系统设计实现

摘要：手势作为人类通用的一种肢体语言，在日常的交流中起着重要的作用，如何进行手势识别是当前人机交互领域的研究热点。基于手部关键点检测的增强现实交互系统是一种将手部姿态检测技术与增强现实技术相结合的交互系统。该系统可以实现通过手部姿态控制虚拟物体的运动、调整虚拟物体的位置，为用户提供更加直观、自然的交互体验。该系统由手部检测、手部关键点检测、手部追踪、手势识别、操作逻辑及应用五部分组成。本文将从这五部分以及系统的需求分析、开发技术分析来阐述该系统的设计与实现。

关键词：手部检测；关键点检测；手势识别；增强现实

The Design and Implementation of Hand Key Point Detection-based Augmented Reality Interaction System

Abstract: Gestures, as a universal form of body language, play a significant role in everyday communication for humans. The study of gesture recognition is currently a hot topic in the field of human-computer interaction. The hand gesture-based augmented reality interaction system is an interactive system that combines hand posture detection technology with augmented reality technology. This system can achieve operations such as controlling the movement and position of virtual objects through hand gestures, providing users with a more intuitive and natural interaction experience. The system consists of five parts: hand detection, hand key point detection, hand tracking, gesture recognition, operation logic, and application. This article will elaborate on the design and implementation of the system based on these five sections, as well as the requirements analysis and development technology analysis of the system.

Key words: Hand detection; key point detection; gesture recognition; augmented reality.

目 录

1	绪 论	1
1.1	研究背景及发展趋势	1
1.2	系统开发的目的是意义	1
1.3	技术背景	2
2	系统需求分析	4
2.1	系统可行性分析	4
2.1.1	系统可行性	4
2.1.2	时间可行性	4
2.2	系统需求分析	5
2.2.1	系统资源需求	5
2.2.2	功能需求分析	5
3	系统开发技术分析	6
3.1	YOLOv5	6
3.2	ResNet50	6
3.3	OpenCV	7
3.4	NVIDIA CUDA	7
3.5	Git	8
4	系统实现	10
4.1	手部检测	11
4.1.1	关键代码	11
4.1.2	实现效果	12
4.2	关键点检测	12
4.2.1	数据集	13
4.2.2	关键代码	13
4.2.3	实现效果	15
4.3	手部追踪	15
4.3.1	计算 IoU	16

4.3.2	手势轨迹	17
4.3.3	关键代码	18
4.3.4	实现效果	20
4.4	手势识别	21
4.4.1	手势识别方法的演变	22
4.4.2	手势识别原理	22
4.4.3	不同的向量选取策略	26
4.4.4	不同策略的比较	28
4.4.5	关键代码	29
4.4.6	实现效果	30
4.5	操作逻辑及应用	32
4.5.1	交互逻辑	32
4.5.2	应用开发	33
4.5.3	菜单界面	33
4.5.4	图像内容识别应用	35
4.5.5	虚拟键盘应用	38
4.5.6	虚拟钢琴应用	39
5	结论与展望	41
	参考文献	42
	致谢	44

1 绪 论

1.1 研究背景及发展趋势

传统的增强现实交互方式往往需要用户通过鼠标、键盘等外部设备进行操作，而手部关键点检测技术可以更加自然地捕捉用户的手部动作，提高用户的交互体验，提供了一种非接触式控制的方法^[1]。未来的生活场景朝着越来越智能化的方向发展，智能家居、自动驾驶、智慧医疗，第一视角沉浸式交互等应用场景，都将离不开手势交互的身影，无论是第一视角手势交互还是第三视角手势交互，其发展潜力都是无穷的^[2]。

目前，基于手部关键点检测的增强现实交互系统的研究水平已经相当成熟。基于手势识别的人机交互界面有两类：一类是由手势来操作虚拟的对象，目前有呈现在计算机上的图片、游戏、Windows 操作系统中的窗口等。美国 CMU 研制的机器人就可以识别用户的手臂做出的手势来执行相应的动作。另一类指手势直接作为人和计算机直接交流的语言工具。美国的手语识别系统可以将手势翻译为语音，对广大残疾朋友有莫大帮助。2012 年，国内彩电康佳公司的 X8100 系列是国内首款运用手势来识别电视的，通过上面的摄像头来识别人类的手势，手势的水平方向运动可以用于调台和翻页，轻拍动作可以选择和确定，手部运动控制指示着鼠标的移动^[3]。

研究者们提出了许多有效的手部关键点检测算法和手部姿态识别算法，并将这些算法应用于增强现实交互系统的设计和实现中。然而，目前基于手部关键点检测的增强现实交互系统还存在一些问题。首先，手部姿态识别的准确率和速度需要进一步提高。其次，增强现实技术的应用还需要更加灵活和自然。最后，基于手部关键点检测的增强现实交互系统的应用场景和功能还有待进一步探索和拓展。

1.2 系统开发的目的是和意义

随着计算机技术的不断改进与发展，替代原来的键盘和鼠标，将手势识别应用于与计算机的交流上，将带来人机交互的全新发展^[3]。也随着计算机视觉和深度学习技术的不断发展，基于手部关键点检测的增强现实交互系统逐渐成为研究热点。该系统可以为用户提供更加直观、自然的交互方式，具有广泛的应用前景。例如，在教育、医疗、娱乐等领域中，基于手部关键点检测的增强现实交互系统可以为用户提供更加生动、有趣的交互体验，提高用户的参与度和体验感。

该系统提供了自然而直观的用户交互，该系统通过识别和追踪手部关键点以及手势识别，使用户能够直接通过手部动作与虚拟对象进行交互，而无需依赖传统的键盘、鼠

标或控制器。这种自然的交互方式能够提高用户的沉浸感和操作体验，使用户能够更直观地控制和操作虚拟环境。

该系统是增强现实应用的扩展，通过结合手部关键点检测和增强现实技术，该系统可以将虚拟对象与现实世界相结合，创造出更具沉浸感和互动性的增强现实体验。用户可以使用手势来控制 and 操作虚拟对象，例如旋转、缩放、移动等，从而使增强现实应用更加灵活和多样化。

该系统提高了交互的灵活性和便利性，相比传统的交互方式，如键盘和鼠标，手势交互具有更高的灵活性和自由度。用户可以通过简单的手势动作完成复杂的操作，使交互过程更加直观和自然。此外，手势交互还可以提供更加便捷的操作方式，特别适用于一些需要快速反应和动态交互的场景，如游戏、虚拟现实训练等。

该系统的开发探索了基于深度学习和计算机视觉技术的手部关键点检测、手势识别和追踪等领域。通过将这些技术应用于实际系统开发中，推动了相关领域的研究和应用进步，为未来的交互技术发展提供了有益的经验 and 参考。

总之，这个基于手部关键点检测的增强现实交互系统的目标和意义在于提供一种自然、直观、灵活的用户交互方式，并推动相关技术的发展，为增强现实应用领域带来更好的用户体验和交互效果。

1.3 技术背景

近年来，手部检测和基于手部检测的行为分析已成为计算机视觉领域备受关注的研究方向之一^[4]。随着深度学习的快速发展，基于深度学习的手部检测和行为分析方法取得了显著的进展。卷积神经网络在特征学习方面的优越性不断提高，这也使得手部检测和行为分析的准确性得到了明显提升。基于手部关键点检测的增强现实交互系统正是计算机视觉和增强现实技术相结合的产物。

在计算机视觉领域，研究者们提出了多种有效的手部检测、手部关键点检测和手部姿态识别算法。其中包括基于卷积神经网络（CNN）和深度神经网络（DNN）的方法等。这些算法通过对手部图像进行分析和学习，能够准确地检测手部位置、关键点位置以及手势姿态。而在增强现实领域，研究者们也提出了多种有效的增强现实技术，如基于标记的增强现实技术和基于视觉里程计（VIO）的增强现实技术等。

在 AR/VR 的使用过程中，手部检测和追踪对于提供一种自然的交互方式起着重要的作用^[5]。通过识别和追踪手部动作和手势，系统能够理解用户的意图，并将其映射到虚拟环境中，从而实现更直观、灵活的交互体验。用户可以通过简单的手势动作来控制 and 操作虚拟对象，与虚拟环境进行互动。这种自然的交互方式使得用户能够更好地融入虚拟世界，增加了交互的乐趣和沉浸感。

本文所实现的系统，首先使用基于 YOLOv5 目标检测算法的手部检测获得图像中的手部图像，将手部图像输入到下一个环节；使用基于深度学习的关键点检测标记出手部关键点；计算相邻帧数图像中手部 bounding box 的 IoU 值来进行手部追踪；通过手部相关关节的角度信息识别出手势。

2 系统需求分析

2.1 系统可行性分析

2.1.1 系统可行性

基于手部关键点检测的增强现实交互系统在技术上是可行的。手部关键点检测算法和深度学习技术的发展提供了准确检测和跟踪手部的方法。本文系统采用 Python 编写。在手部检测功能方面，本文系统使用 YOLOv5 目标检测算法，以此来对图像中的手部目标进行定位。在关键点检测方面，本文系统采用基于深度学习的手部关键点检测的模型（如 ResNet50），该模型需要将手部检测的部分图像输入，采集到手部的关键点，包括可以精准定位手部的 21 个主要关节点、各指尖的坐标信息。对于手部追踪方面，本文系统基于 IoU 的计算进行手部轨迹的追踪。对于手势识别，本文系统采用了基于手部骨架的手势识别，利用检测出的关键点得到手部姿态的信息，如各个关节之间的角度，以此进行手势的识别。关于系统内的操作逻辑以及应用的开发，系统采用了基于手势识别的交互逻辑设计。交互逻辑方面，系统中不同手势可以被识别为不同的操作，例如“点击”、“长按”等。利用这些交互逻辑，本文系统开发了图像内容识别、虚拟键盘、虚拟钢琴等应用。同时，相关技术库和工具（如 OpenCV 和深度学习框架）的存在使得系统的开发和实现更加可行，也使得系统可以进行图像处理、摄像头输入等功能。

2.1.2 时间可行性

本基于关键点检测的增强现实交互系统在开发之前进行了严格的计划和时间表的制定，在系统开发之前，制定了详细的计划和时间表。通过定义清晰的里程碑和任务分解，确保了开发过程中的时间控制和任务管理。这有助于准确估计开发所需的时间，并提前发现和解决潜在的时间冲突。在系统开发之前，进行了大量的调查和资料搜索。通过对相关技术和领域的研究，收集了关于手部关键点检测、增强现实交互等方面的资料。这些准备工作为后续的系统设计和开发提供了重要的参考和指导，节省了开发过程中的时间。进行了用户调查和需求分析。通过与目标用户的沟通和调查，了解了用户的期望和需求。这有助于明确系统的功能和交互要求，并避免在开发过程中出现方向偏差和需求变更，节省了重复开发和修改的时间。系统采用了迭代开发和测试的方法，充分使用 github 的功能。通过将开发过程分为多个阶段和迭代周期，可以及时发现和修复问题，确保系统在早期阶段就能够达到预期的功能和性能要求。这种敏捷的开发方式可以节省

系统开发的时间，并提高系统的质量和稳定性。

2.2 系统需求分析

2.2.1 系统资源需求

系统开发所需的硬件和软件资源是可行的。本文系统采用 PyCharm Community 进行开发。系统需要适当的计算机设备，如一定配置的计算机，由于该系统需要对实时画面进行处理，计算机需要较好的 CPU 处理器以及显卡，否则实时画面的帧率会降低，以及能提供稳定视频流的摄像头，还需要稳定的网络连接来支持系统的运行。此外，系统还需要足够的存储空间来存储相关数据和模型。

2.2.2 功能需求分析

本基于关键点检测的增强现实交互系统的功能需求的确定是在大量的基础调查之上完成的，初步设计了本基于关键点检测的增强现实交互系统的基本功能需求，最后在指导老师的指导下完成了本基于关键点检测的增强现实交互系统的功能模块设计。具体的主要功能包括：手部检测模块，手部关键点检测模块，手部追踪模块，手势识别模块，应用模块包括图像内容识别应用，虚拟键盘应用，虚拟钢琴应用。

3 系统开发技术分析

3.1 YOLOv5

YOLOv5 是一种高效的目标检测算法，采用了单阶段检测器的架构，能够在图像中高效地检测出多个目标，并同时提供它们的位置和类别信息。相比其前身 YOLOv4，YOLOv5 在检测性能方面取得了显著的改进，同时具备更快的速度。这使得 YOLOv5 成为一种广受欢迎的目标检测网络，具有高精度的检测结果、快速的检测速度和易于移植的特点^[6]。

YOLOv5 的主要改进包括引入了一种新的骨干网络架构——CSPDarkNet，该网络结构在计算效率和检测性能之间取得了良好的平衡。此外，YOLOv5 还采用了更高效的检测头设计，提供了更精确的锚框生成方式，并优化了训练策略和数据增强方法等。这些改进使得 YOLOv5 在目标检测任务中能够取得更好的性能。

由于 YOLOv5 具备高效、准确和易于实现等优点，它在目标检测领域得到了广泛的应用。许多研究者和开发者选择使用 YOLOv5 作为他们的目标检测算法，因为它能够在不同的应用场景中取得优异的表现。此外，YOLOv5 在多个基准测试中展现出领先的性能，成为目标检测领域的一项重要技术。

总的来说，YOLOv5 的引入对于目标检测领域具有重要的意义。它不仅提供了高效且准确的目标检测能力，而且在实际应用中具备较快的速度和易于实现的特点，为各种视觉任务和应用提供了可靠的解决方案。随着技术的不断发展，YOLOv5 有望继续演进并推动目标检测领域的进一步发展。

3.2 ResNet50

ResNet50 (Residual Network-50) 是一种深度卷积神经网络模型，属于 ResNet 系列网络的重要成员，由微软研究院的 Kaiming He 等人于 2015 年提出。该模型主要用于图像分类任务，并在 ImageNet 数据集上取得了较高的准确率。

ResNet50 的主要特点是引入了残差块 (residual block)，以缓解深度网络训练过程中的梯度消失和梯度爆炸问题。通过在输入和输出之间引入跨层连接 (shortcut connection)，残差块可以更有效地传递信息，提高网络的训练效率和准确率。此外，ResNet50 还采用了其他一些技巧，如批归一化 (batch normalization) 和全局平均池化 (global average pooling)，进一步提升了模型的性能。

由于 ResNet50 具有高效的训练和出色的性能，它在计算机视觉领域得到了广泛的

应用，如图像分类、物体检测、图像分割等任务。此外，ResNet50 也成为了深度学习领域的经典模型之一，对于后续网络设计和优化提供了重要的参考。

在本文系统中，关键点检测模块采用了 ResNet50 作为基础模型。通过利用 ResNet50 的卓越性能和特性，该模块能够准确地定位和标记出手部关键点，为后续的增强现实交互提供关键信息和支持。因此，选择 ResNet50 作为关键点检测模块的基础，对于实现精确的手部关键点检测以及提供优质的增强现实交互体验具有重要意义。

3.3 OpenCV

OpenCV（Open Source Computer Vision Library）是一个开源的计算机视觉库，旨在为开发人员提供一个简单易用的工具集，用于处理图像和视频数据。OpenCV 是由英特尔公司在 1999 年创建，并在后来变成了开源项目，现在已经成为了计算机视觉领域最广泛使用的开源库之一。

OpenCV 的应用领域非常广泛，主要涉及以下几个方面：

（1）图像处理：OpenCV 可以用于图像的读取、显示、转换、过滤、特征提取等方面的处理。例如，可以使用 OpenCV 对图像进行边缘检测、模糊、二值化、色彩空间转换等操作。

（2）物体检测和识别：OpenCV 可以用于物体检测和识别的任务，例如使用 Haar 分类器进行人脸检测、使用 HOG+SVM 进行行人检测等。OpenCV 还提供了一些预训练的模型，例如基于深度学习的人脸检测器和物体检测器。

（3）视频处理：OpenCV 可以用于视频的读取、显示、转换、处理和分析。例如，可以使用 OpenCV 进行视频的剪辑、拼接、稳定、背景去除等操作。

（4）机器学习：OpenCV 提供了许多常用的机器学习算法和工具，例如 K 近邻、支持向量机、决策树、随机森林等。这些算法可以用于图像分类、物体识别、目标跟踪等任务。

（5）增强现实：OpenCV 可以用于增强现实的开发。例如，可以使用 OpenCV 进行相机姿态估计、场景重建等操作，从而实现更加逼真的增强现实体验。

除了以上领域，OpenCV 还被广泛应用于机器人、医学图像处理、工业自动化等领域。由于其丰富的功能和易用性，OpenCV 已经成为了计算机视觉领域最受欢迎的工具之一。因此 OpenCV 也是本文系统中图像处理、摄像头输入等功能的关键组件之一。

3.4 NVIDIA CUDA

NVIDIA CUDA（Compute Unified Device Architecture）是一种并行计算平台和编程

模型，用于利用 NVIDIA GPU（Graphics Processing Unit，图形处理器）的强大计算能力进行高性能计算。CUDA 允许开发人员使用 C/C++、Fortran 等编程语言来编写 GPU 加速的应用程序，并通过并行计算来提升计算性能。



图 3.1 CUDA 安装界面

如图 3.1 所示为本系统的 NVIDIA CUDA 安装界面，本文系统安装在使用 MX250 显卡的计算机上，可以使用 NVIDIA CUDA 来提升对实时画面的处理速度，因为本文系统中的实时目标检测和跟踪需要在短时间内对大量图像数据进行处理和分析，CUDA 的并行计算能力可以使目标检测和跟踪算法能够快速处理图像序列，实现实时性能，使本系统能够在实时场景中进行准确的目标检测和跟踪。

3.5 Git

Git 是一种分布式版本控制系统，被广泛用于软件开发中的源代码管理。它由 Linus Torvalds 于 2005 年创建，并成为了许多开源项目和商业项目中的首选工具。

本文系统的开发采用了使用了 Git，它提供了一个强大而灵活的版本控制系统。在开发的过程中可以轻松地管理代码的版本、分支、合并和发布，同时保留详细的历史记录和代码追踪能力。在本文系统的开发过程中，Git 对于开发进程的加快以及系统版本的迭代起到了重要作用。

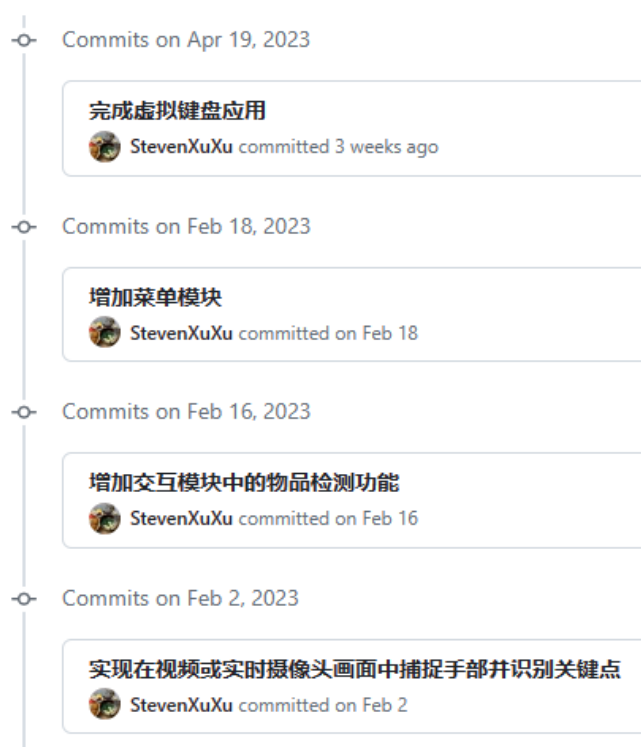


图 3.2 系统版本提交示例

如图 3.2 所示，可以在 GitHub 中看到系统开发过程中不同版本的发布时间以及迭代过程。

4 系统实现

本文系统分为了手部检测，关键点检测，手部追踪，手势识别，操作逻辑及应用及部分组成。

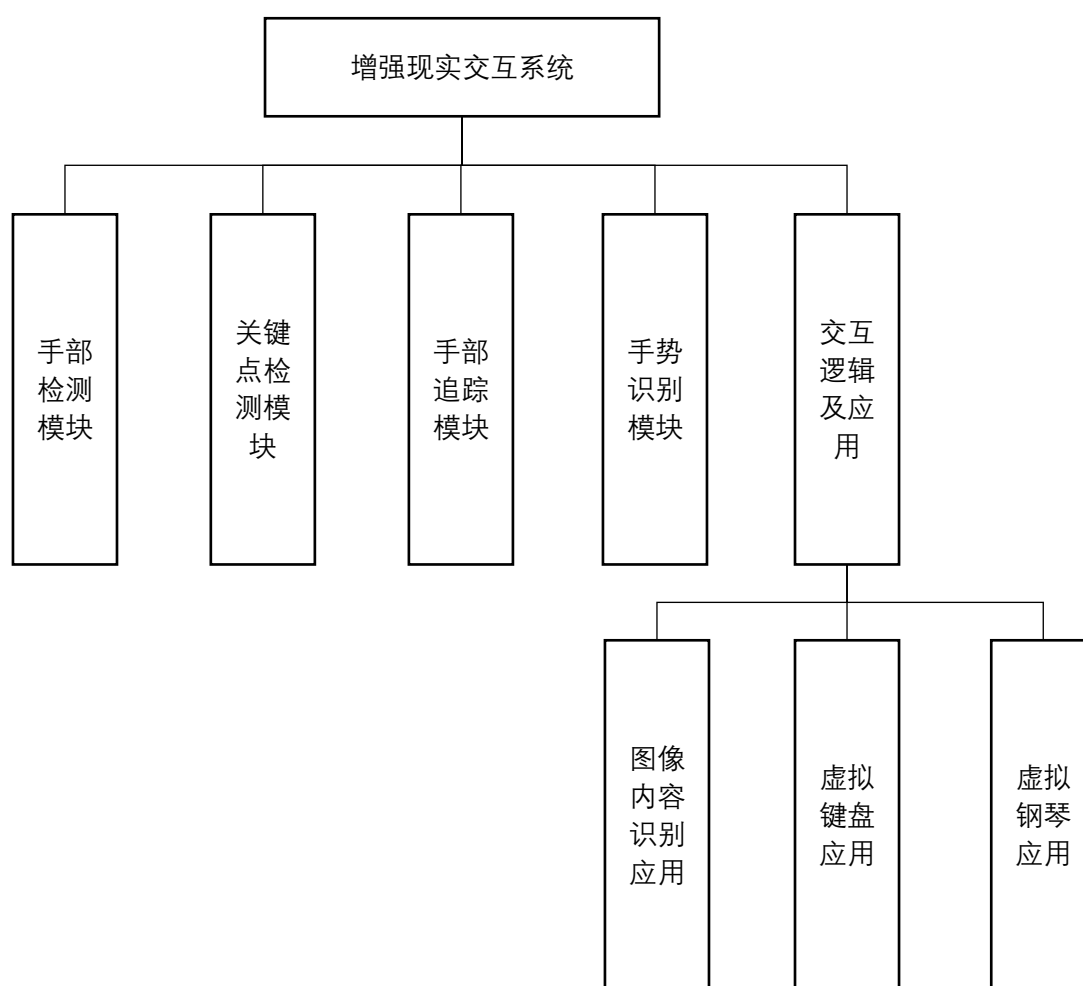


图 4.1 系统总体结构图

图 4.1 所示即为系统总体结构图，其中系统的应用有图像内容识别应用、虚拟键盘应用、虚拟钢琴应用三个。

下面将对系统的各个模块的详细设计进行介绍。

4.1 手部检测

系统中的手部检测模块使用了 YOLOv5 目标检测算法，若以视频作为输入流，则将每一帧图像作为输入，经过手部检测模型后得到一个手部目标框的坐标、长、宽等信息。

4.1.1 关键代码

本文给出使用 YOLOv5 算法来进行手部检测的关键代码，隐去了 YOLOv5 相关模型的一些细节。

```
for frame_idx, (path, img, im0s, vid_cap) in enumerate(dataset):
    img = torch.from_numpy(img).to(device)
    img = img.half() if half else img.float()
    img /= 255.0
    if img.ndimension() == 3:
        img = img.unsqueeze(0)
    t1 = time_synchronized()
    pred = model_yolo5(img, augment=opt.augment)[0]
    pred = non_max_suppression(pred, opt.conf_thres, opt.iou_thres, classes=[0])
```

本文系统对画面每一帧的图片进行了预处理，并将其输入到模型中得到了相应的检测结果。在整个流程中，“dataset”表示数据流，可以为视频流或者摄像头的输入画面。每一帧的画面数据则用“img”表示，我们需要对每一帧的画面数据进行预处理，以使其适合输入到模型中进行处理。在经过预处理后，我们将处理后的画面数据输入到模型中进行处理，并得到相应的检测结果。将画面输入到模型中，再进行极大值抑制，最后的“pred”即为输出的结果，结果标注了每个检测出的 bounding box 的位置信息，包括左上角以及右下角的坐标。这些位置信息可以被用于后续的应用场景中，例如关键点检测、手部追踪等等。

具体而言，预处理过程包括了对图像的缩放、归一化、通道交换等操作。首先，对于输入的图像，我们需要对其进行缩放操作，使其大小适合于模型的输入要求。其次，需要对图像进行归一化处理，即将图像的像素值缩放到 0 到 1 之间。最后，由于模型的输入要求图像通道的顺序为 RGB，而原始图像通道的顺序可能是 BGR 或者其他顺序，因此需要对图像进行通道交换操作。

4.1.2 实现效果

将图片或者视频流的一帧画面输入到使用 YOLOv5 算法的模型中，即可得到检测到的手部 bounding box 的坐标、宽度、长度等信息，使用 cv2 标注在图中即可。

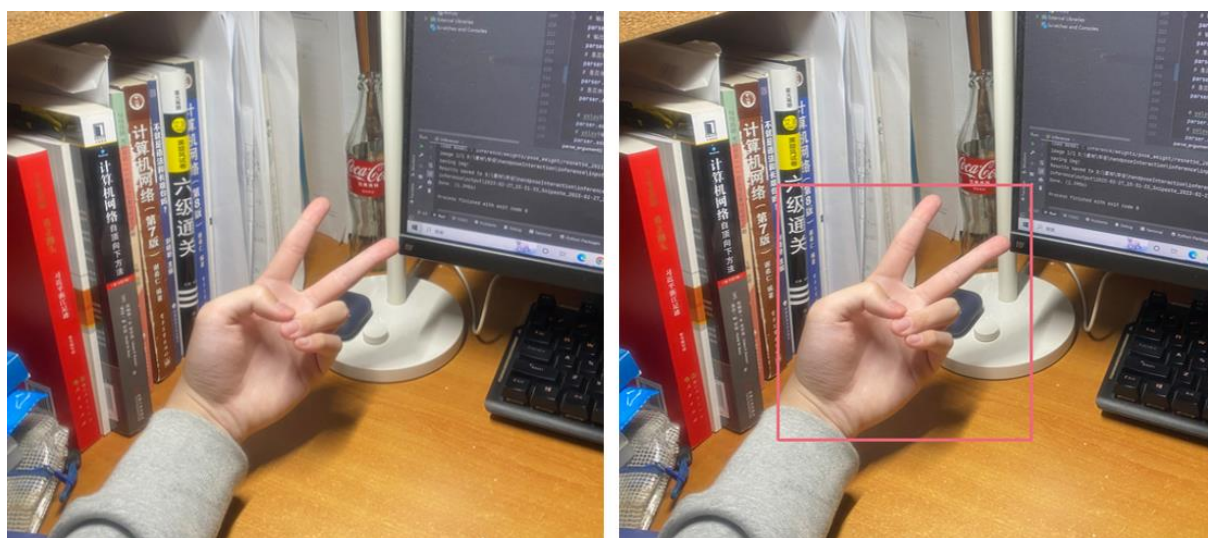


图 4.2 手部检测实现效果

如图 4.2 所示，可以看到最终实现效果，其中隐去了文字标注。然后将图像中的 bounding box 范围截取出来作为关键点检测的输入，从而提高检测的准确率。

之所以需要先进行手部检测，是要给关键点检测模型提供一个相对来说大小合适、内容准确的图像以提高关键点检测的准确率。此外，有了手部检测得到的 bounding box 信息才能实现对特定手部的追踪，基于这一点本文就可以在视频流检测并且有多只手的情况下区分每一个目标，进行手部追踪。

4.2 关键点检测

本文采用基于深度学习的手部关键点检测的模型，该模型需要将手部检测的部分图像输入，采集到手部的关键点，包括可以精准定位手部的 21 个主要关节点、各指尖的坐标信息^[7]。有些模型的训练数据还能够区分出手指的深度信息^[8]，即各个关键点距离摄像头的远近，本文系统的关键点检测模型所使用的数据集中每个关键点的信息是二维的，包括 x 轴和 y 轴的坐标。

4.2.1 数据集

该模型采用了 `handpose_datasets_v1` 数据集^[9]，该数据集包括网络图片及数据集 <<Large-scale Multiview 3D Hand Pose Dataset>> 筛选动作重复度低的部分图片，进行制作，共 49062 个样本。



图 4.3 标注样例

如图 4.3 所示，数据集标注了手部的关键点坐标，其中定位了手部各重要关节、指尖的位置。

4.2.2 关键代码

手部关键点检测模块是本文所述系统中的重要组成部分，它能够对手部图像进行关键点检测，进而实现手势识别等功能。在本文中，我们展示了该模块的关键代码，其中隐去了相关检测模型的具体实现。

```

if det_box is not None and len(det_box):
    det_box[:, :4] = scale_coords(img.shape[2:], det_box[:, :4], im0.shape).round()
    s += '%g %ss ' % (len(det_box), names[0])
    keypoint_list = []
    for one_hand_box in det_box.data.cpu().numpy():
        cut_img = im0[int(one_hand_box[1]):int(one_hand_box[3]),
                       int(one_hand_box[0]):int(one_hand_box[2])]
        key_img = cv2.resize(cut_img, (res50_img_size[1], res50_img_size[0]),
                             interpolation=cv2.INTER_CUBIC)
        key_img = (key_img.astype(np.float32) - 128.) / 256.
        key_img = torch.from_numpy(key_img.transpose(2, 0, 1)).unsqueeze_(0)
        if torch.cuda.is_available():
            key_img = key_img.cuda()
        key_img = key_img.half() if half else key_img.float()
        key_output = model_res50(key_img)
        key_output = key_output.cpu().detach().numpy()
        key_output = np.squeeze(key_output)
        hand_ = []
        for i in range(int(key_output.shape[0] / 2)):
            x = (key_output[i * 2 + 0] * float(cut_img.shape[1])) + int(one_hand_box[0])
            y = (key_output[i * 2 + 1] * float(cut_img.shape[0])) + int(one_hand_box[1])
            hand_.append([x, y])
        keypoint_list.append(hand_)

```

以上即为关键点检测的关键代码，关键点检测步骤包括了多个子步骤。首先，手部图像需要被切割出来，其中“det_box”即为。接下来，对切割出来的图像进行缩放、归一化等操作，以便于将其输入到检测模型中。这些操作包括图像的裁剪、大小缩放、均值标准化等预处理步骤，可以有效地提高检测模型的准确率和鲁棒性。最后，检测模型会输出手部图像中每个关键点的坐标信息，这些信息被存储在“keypoint_list”数组中。这些关键点信息可以用于手势识别、手部姿态估计等任务中，为后续的手部动作识别和交互提供重要的基础数据。

4.2.3 实现效果

在得到了 21 个关键点的坐标以后，根据手指状态将关键点自然地相连，可以得到一个很直观的手势状态图，或者称为手部骨架图。有了手部骨架，就可以后面的步骤中很方便地得到手部状态，如手势、指尖位置等。

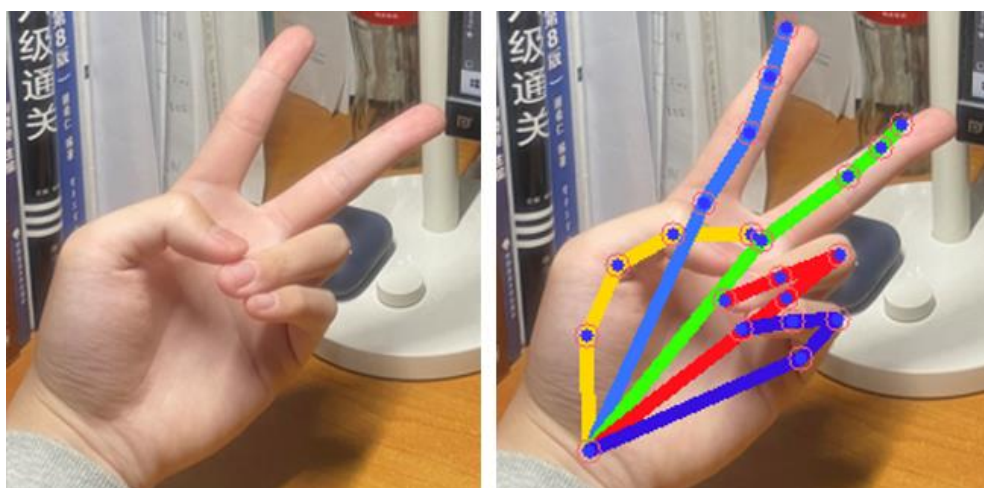


图 4.4 关键点检测实现效果

如图 4.4 所示，将关键点连接以后可以很直观地表现出手势状态。当然，关键点的连接方式并不唯一，本文只是提供了其中一种方案，可以根据需求来更改。

4.3 手部追踪

手部追踪对于本文的系统十分重要，手部追踪技术被引入到基于手部关键点检测的增强现实交互系统中，以提高系统的交互性和稳定性。本文所使用的是基于手部检测的手部追踪，通过计算手部检测所得到的 bounding box 的 IoU 来比较相邻帧图像之间的手部图像的关系，从而确定手部单位的移动位置。手部追踪可以为用户提供更加自然和直观的交互方式。通过手部追踪技术，用户可以实时地控制虚拟物体的位置、旋转和缩放等操作，并且可以根据手部动作的变化来触发不同的交互功能。

此外，手部追踪技术还可以为增强现实交互系统提供更加稳定和可靠的性能。由于手部追踪技术可以更准确地跟踪手部动作，因此可以避免由于光线、遮挡和手部姿态变化等因素导致的误差和不稳定性。这可以提高交互系统的鲁棒性和用户体验，并且为用户提供更加流畅和自然的交互体验。

4.3.1 计算 IoU

目标检测当中，有一个常用的指标，叫 IoU（Intersection over Union）或交并比，它常常用来衡量目标检测任务中，预测结果的位置信息的准确程度。在目标跟踪任务中，IoU 可以被用来衡量当前目标跟踪框与前一帧中目标跟踪框之间的相似度。如果 IoU 值足够高，就可以认为当前目标跟踪框对应的是前一帧中的同一目标，本文基于这一原理，就可以实现对于手部的追踪。

IoU 的计算原理非常简单，如公式（4.1）所示。

$$IoU = (Area\ of\ Overlap) / (Area\ of\ Union) \quad (4.1)$$

用数学中集合的语言来说，也就是两个区域的“交集”的面积，除以两个区域的“并集”的面积。

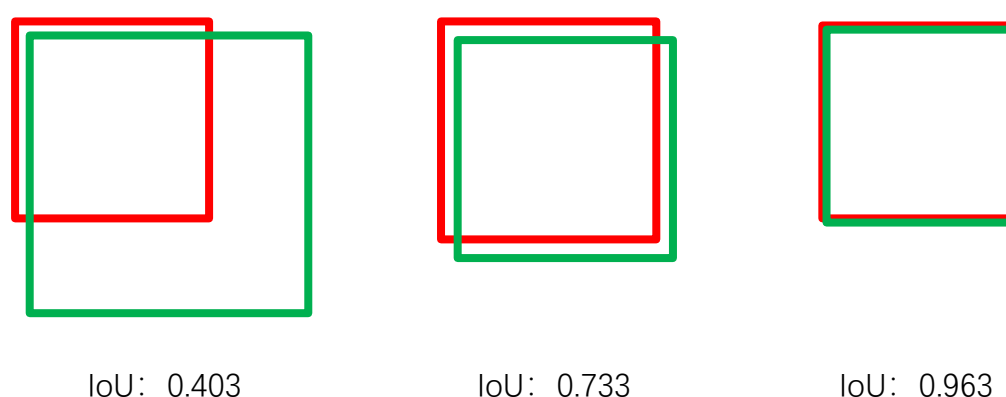


图 4.5 不同 IoU 值示例

图 4.5 展示了三种不同的 IoU 值，其中红色框和绿色框分别为上一帧和当前帧的 bounding box，图中从左到右的 IoU 值逐渐增大，也就是说更能相信绿色框是从红色框改变而来的，即它们是同一个目标在不同帧图像下的位置。

本文利用 IoU 实现了对手部的追踪，使得该系统可以在多目标的情况下，对不同的目标独立进行追踪。有了这个功能，使增强现实的交互更加流畅，对于操作逻辑的开发也十分重要，因为可以在此基础上得到用户的手势轨迹，由手部轨迹来判断用户的意图，如“下拉”操作，例如在 360 浏览器中，“下拉右转”表示“返回”操作。本文认为类似的操作逻辑有很大的创新利用空间。

4.3.2 手势轨迹

手势轨迹对于本文的系统十分重要，有了手势轨迹才能对用户的如“下划”、“上划”等动态手势进行识别与定义。利用对 IoU 值的分析，本文系统可以实现在对目标情况下对不同目标进行独立追踪，同时也给了手势轨迹的实现提供了可能性，只有对目标独立，才能进行手势轨迹的计算。本文系统的 tracker 模块的其中一个功能就是记录用户的手势轨迹。

首先要确定一个绘制点，类似于画笔，这样才能确定用户想要点击、移动的具体位置，本文所选取的是食指指尖位置。由于手部检测以及关键点检测的精度问题，在每一帧画面之间的识别位置可能不统一，会出现检测出的关键点“抖动”的现象，因此系统要确定一个阈值，只有指尖位置改变量大于这个阈值，才能判定这是用户做出的行为，而不是“抖动”所造成的假象。此外，还需要设置一个触发“绘画”的手势，以此来区分用户是否想要进行交互，这一点需要符合用户自然的操作规律，本文选择的是图 4.10 中的 1、2、3 号手势。

在用户做出交互动作之后，即开始记录用户的食指指尖的位置信息，存在一组数组中，本文系统所定义的基础方向有“上”“下”“左”“右”四种，每一帧还需要对用户的手势轨迹进行分析，从而发现是否是系统已经定义的操作，若是则做出响应。手势追踪的效果如图 4.3 所示。

4.3.3 关键代码

本文系统中计算两帧画面的 bounding box 的 IoU 值的函数的代码如下：

```
def __compute_iou(self, x1, y1, x2, y2):
    box1 = np.array([x1, y1, x2, y2])
    iou_list = []
    for box2 in self.last_box:
        h = max(0, min(box1[2], box2[2]) - max(box1[0], box2[0]))
        w = max(0, min(box1[3], box2[3]) - max(box1[1], box2[1]))
        area_box1 = ((box1[2] - box1[0]) * (box1[3] - box1[1]))
        area_box2 = ((box2[2] - box2[0]) * (box2[3] - box2[1]))
        inter = w * h
        union = area_box1 + area_box2 - inter
        iou = inter / union
        iou_list.append(iou)
    iou_list = np.array(iou_list)
    return iou_list
```

其中输入的“x1”、“y1”、“x2”、“y2”分别为第一个 bounding box 的左上角的 x 轴、y 轴坐标和第二个 bounding box 的左上角的 x 轴、y 轴坐标。

根据 IoU 更新当前帧的手部信息、实现手部追踪的代码如下：

```
def update(self, det, key_point_list):
    for n, (*xyxy, conf, cls) in enumerate(det):
        x1, y1, x2, y2 = xyxy[0].item(), xyxy[1].item(), xyxy[2].item(), xyxy[3].item()
        iou: np.ndarray = self.__compute_iou(x1, y1, x2, y2)
        track_id, iou_val = iou.argmax(), iou.max()
        pose: int = self.__compute_pose(key_point_list[n])
        if iou_val == 0:
            if self.last_box[track_id].max() != 0:
                if self.last_box[1 - track_id].max() != 0:
                    self.update_nodata([0, 1])
                    return
                else:
                    track_id = 1 - track_id
            self.no_data_now = 0
            self.last_box[track_id] = np.array([x1, y1, x2, y2])
            self.active_click[track_id][0] = key_point_list[n][8][0]
            self.active_click[track_id][1] = key_point_list[n][8][1]
            self.active_click[track_id][2] = pose
            self.plot_cache[track_id][0] = np.array([x1, y1, x2, y2, conf, track_id, iou_val,
            pose], dtype=np.float32)
            self.plot_cache[track_id][1] = key_point_list[n]
            if len(det) == 1:
                self.update_nodata(1 - track_id, now=True)
```

其中对可能有两只手进行遍历，获得一个 IoU 数组，分别对应两只手 bounding box 的 IoU 值，“__compute_pose”函数是用来识别当前手的手势，在本文第 5 章会提到。如果当前手的 bounding box 的 IoU 值和两只手都匹配不上，并且当前两只手都有记录，则可认为该情况为手移动过快导致目标丢失，将两只手都重置，“update_nodata”就是用来重置手部信息的。“active_click”数组用来存储每只手的食指坐标信息，这对操作逻辑以及应用的开发非常重要。

4.3.4 实现效果

为了优化 AR 等使用环境下的用户体验，本文系统采用了一些限制措施。考虑到在 AR 等使用环境下，用户通常只会单独操作，因此本系统限制了同时操作的用户数量不会超过一个人。为了进一步提高系统的效率，本文系统还限制最多只能识别两只手及其操作。这样可以减少系统对多个手部动作的识别和处理，从而加快系统响应速度，提高用户体验。同时，这种限制也降低了系统操作逻辑的复杂性，使得系统更加易于使用和维护。总之，本文系统通过限制操作手数和用户数量，旨在为 AR 等使用环境下的用户提供更加高效和便捷的操作体验。



图 4.6 手部追踪实现效果

根据图 4.6 所示，该系统的手部追踪模块采用了一些限制措施来实现最佳的追踪效果。首先，该系统仅限于对一个人的两只手进行实时追踪，这样可以避免对多个人手部动作的干扰。此外，对于两只手的追踪，该系统采用了独立追踪的方式，为每只手分配了一个唯一的编号，例如图中的“id:0”和“id:1”。这种编号信息可以帮助系统准确地追踪每只手的动作，避免混淆和误判。在图中，还可以看到两只手检测出的 bounding box 的 IoU 值都为“1.0”，这意味着两只手都处于静止状态，并且相对于上一帧的位置没有移动过。这种静态状态对于手部追踪模块来说非常重要，因为它可以提高追踪的准确性和稳定性。通过监测手部的位置变化，系统可以实时更新手部的状态信息，从而更好地响应用户的操作。综上所述，该系统的手部追踪模块采用了一些有效的技术手段，以实

现最佳的追踪效果和用户体验。

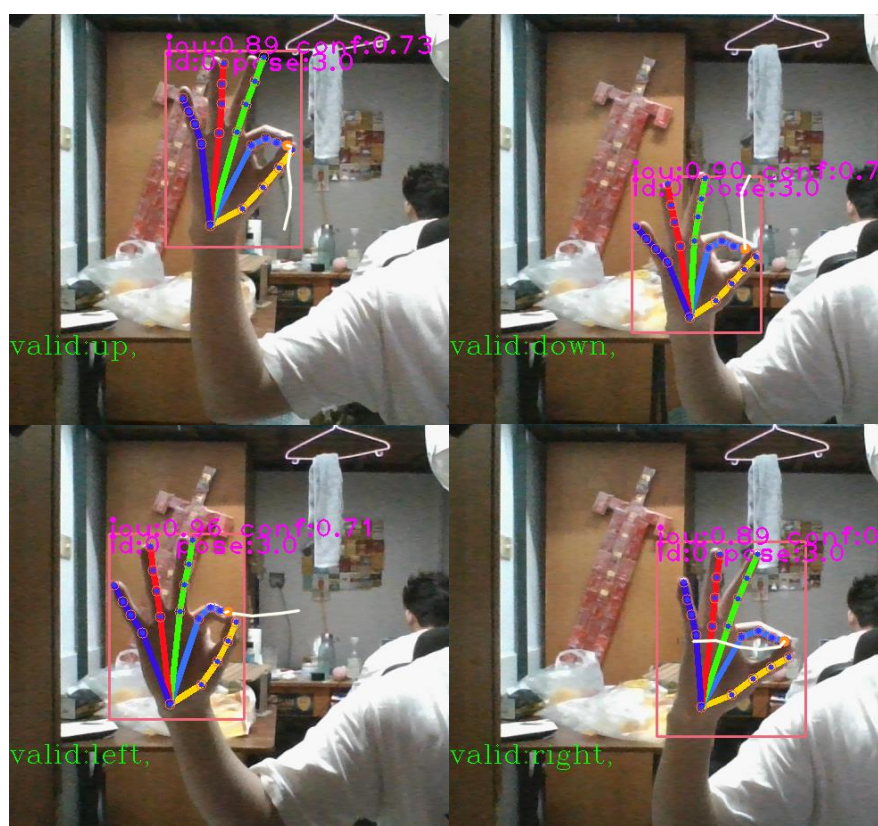


图 4.7 手势追踪实现效果

图 4.7 为手势轨迹的实现效果，可以看到系统可以根据用户的食指指尖做出响应，并绘制出其运行轨迹，也就是图中的白色线段，左下角的绿色字为识别出的轨迹方向，分别为本文系统所定义的基础方向“上”“下”“左”“右”四种，还可以根据需要进行方向的组合，例如可以定义“下右”等响应轨迹。

4.4 手势识别

随着人工智能和计算机视觉发展的发展，在人机交互^[10]，康复医学^[11]，自动驾驶^[12]，手语识别^[13]，手势交互^[14]，虚拟现实^[15]等领域中，基于手的非接触交互方式已经成为许多计算机领域的研究热点。而手势识别在这其中起着重要的作用。一个良好的手势识别模块，可以给用户带来更加自然舒适的体验，也可以增加系统的效率。对于本文的系统，手势识别也起着关键作用。

4.4.1 手势识别方法的演变

早期的手势识别研究是基于数据手套等可穿戴设备实现，利用可穿戴设备的传感器捕捉手势及关节点的位置和角度信息进而实现手势识别。常用的方法有数据手套^[16]和手部标签标记^[17]，数字手套是一种常用的手势识别设备，它通过在手部戴上传感器来捕捉手部运动的数据，再通过算法进行分析和处理，以达到识别手势的目的。然而，数字手套也存在一些弊端：数字手套需要手部穿戴传感器，而传感器的数量通常较多，因此数字手套可能会比较笨重，不太适合长时间佩戴；数字手套的制造成本较高，因为它需要大量的传感器和处理器等硬件设备，并且需要进行复杂的信号处理和算法开发。

到了深度学习时代，深度卷积神经网络（CNN）的出现。深度学习技术的发展使得手势识别的准确率大幅提高。CNN 是一种特殊的神经网络，可以通过学习图像特征来自动提取手势的特征，从而实现高效的手势识别。例如，Zhai 等人在 2021 年对基于深度学习的高效手部检测和手势识别方法研究中，提出了一种基于轻量级网络 SqueezeNet 的手部检测和手势识别模型 SF-FCNet^[18]。

4.4.2 手势识别原理

本文的手势识别采用了基于手部骨架的手势识别，利用检测出的关键点得到手部姿态的信息，如各个关节之间的角度，以此进行手势的识别。相较于使用穿戴设备或者深度学习进行手势识别，该方法成本更低、实时性更好^[7]，而且所需要的数据数量非常少，最少仅需要两只手的正反两面四张图片，正因为这一特点，用户自己就可以很方便快捷地对手势进行定义，可以增加新的手势信息。

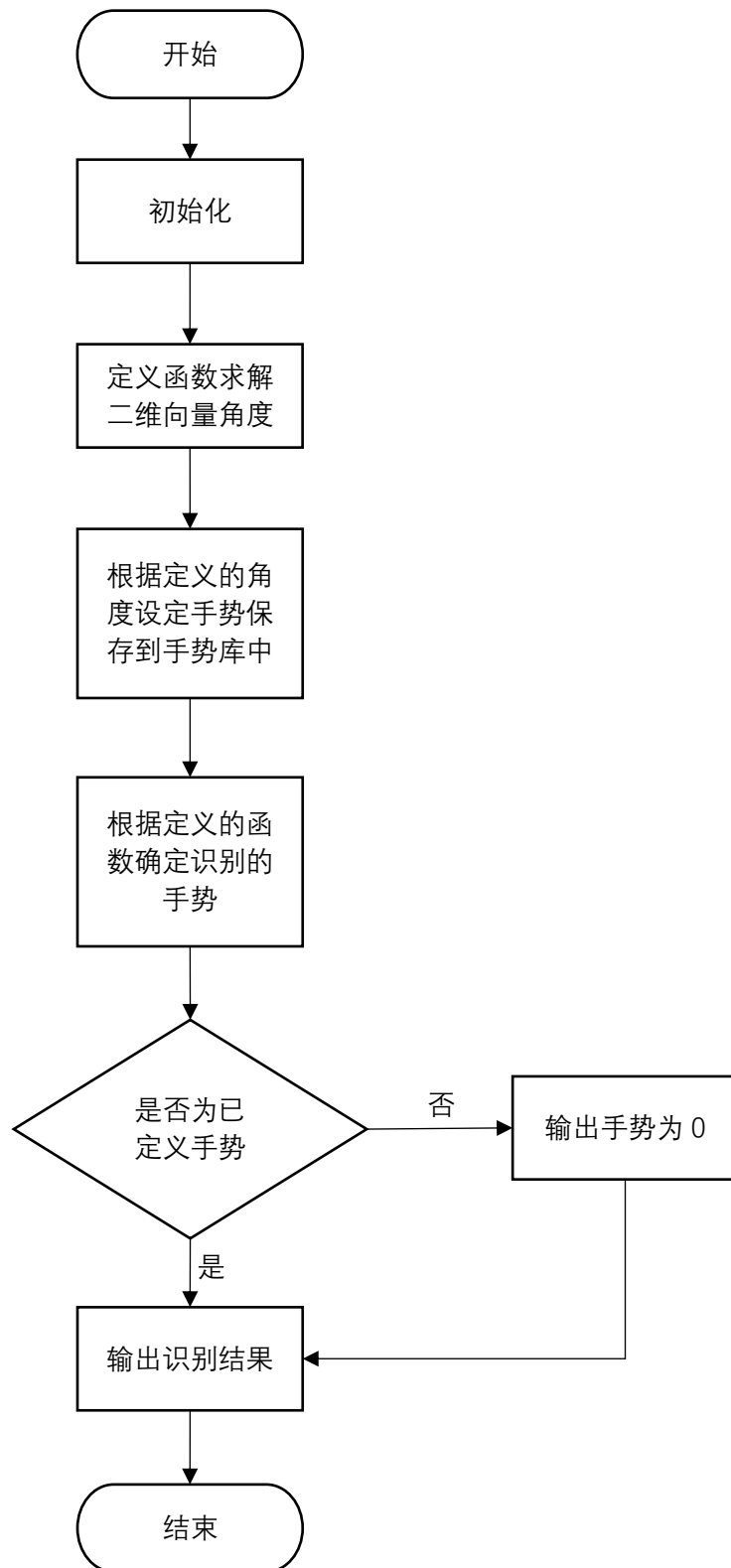


图 4.8 手势识别流程图

如图 4.8 所示，先将自己要定义的手势的图片放入相关文件夹中，启动系统中的 `handpose_setter` 程序，系统会自动对每张图片进行手部检测、关键点检测，通过关键点计算出相关手指关节的角度信息，并将信息存入系统的手势库中。在进行手势识别的过程中，也通过手部检测、关键点检测等步骤用同样的方法计算出需要的角度信息，再将其与手势库中的信息进行比对，找出最匹配的手势即可。

该方法的优点是不需要大量的数据集去事先训练一个模型，在仅有四张图片的情况下就拥有比较好的识别效果，特别是对于手指弯曲角度较小、手部形状变化较小的手势识别效果非常好，并且识别速度快，所需要的算力很少，但是识别的精度依赖于关键点检测模块的检测精度，所以对于一些手指变化角度较大、较为复杂的手势的识别能力有限，在这些情况下相较于深度学习的方法精准度略低。

接下来根据本文的方法，要解决的是如何找出手势库中最匹配的手势。本文这样定义手势之间的特征距离^[7]，通过公式（4.2）计算。

$$D_M = \sum_{i=1}^n \omega_i |\varphi_i - G_i| \quad (4.2)$$

其中 D_M 表示要识别的手势与手势库中的某个手势的特征距离。 n 表示手势的特征的个数，也就是每个手势计算出的 $\cos$ 值的个数，具体要根据所选择的向量选择策略来确定。 ω_i 表示每个距离分量的权重，例如在大多数情况下，大拇指、食指、中指相较于无名指和小拇指来说，对于确定一个手势起着更加重要的作用，因此我们可以增大他们的权重。 G_i 表示要识别的手势的每个距离分量，也就是所需要的每个 $\cos$ 值。 φ_i 即表示在手势库中所存的每个手势的每个距离分量。

在实际应用中，本文还采用了另一种方案，如公式（4.3）所示。

$$S_M = \sum_{i=1}^n [\varphi_i - \alpha \leq G_i \leq \varphi_i + \alpha] \quad (4.3)$$

其中 G_i 表示要识别的手势的每个距离分量，也就是所需要的每个 $\cos$ 值。 φ_i 即表示在手势库中所存的每个手势的每个距离分量。 n 表示手势的特征的个数。 α 表示系统设定的阈值，在该阈值的范围内我们都可以认为他们是相等的。 S_M 即表示在允许的误差范围内的参数的个数。若 S_M 和 n 相等，我们就可以认为所比较的两个手势是相同的。

第二种方案与第一种方案相比，计算速度更加快，因为不必和手势库中的所有手势比较完才能确定手势是否相匹配，但是在有多种手势的参数相近的情况下，使用第一种方案准确率更高。

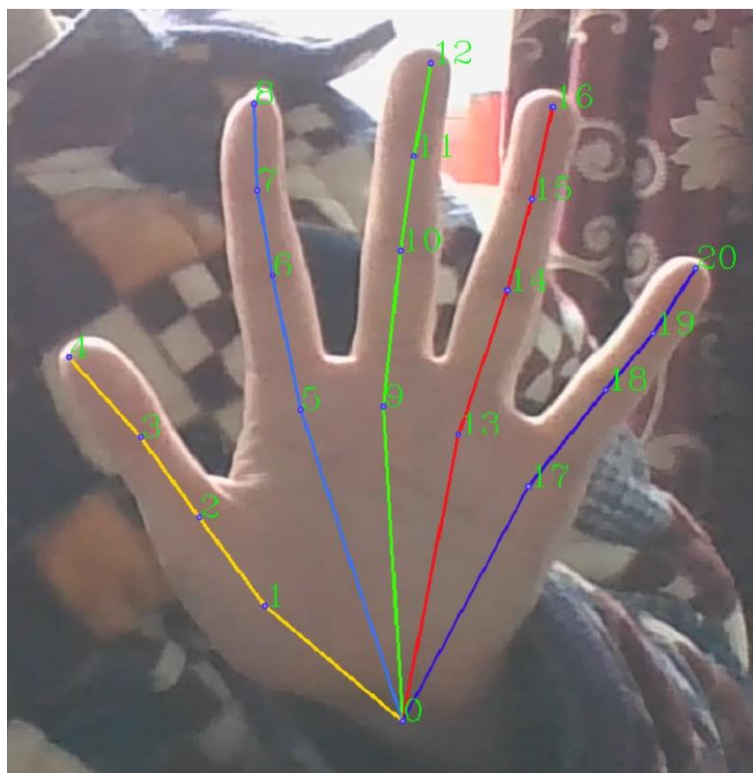


图 4.9 关键点编号

图 4.9 中显示了各个关键点的编号，同时将关键点根据手部骨架状态自然相连。可以看出每一根手指的状态。

根据手部骨架，本文通过计算相关手指的多个二维向量的角度来表示某一个手势。该方法的识别准确率的关键在于对手部骨架中向量的选择。好的向量选择策略可以准确区分不同手势的各个手指的弯曲程度，从而能够在多个角度判断出具体的手势。具体来说，根据手部骨架得到想要的向量，然后计算相关向量的夹角的 $\cos$ 值存入手势库中，在需要识别的时候，通过相同方法计算相关向量夹角的 $\cos$ 值与手势库中的值进行比对，最相近的那一个手势即可作为识别结果。

计算两向量夹角的 $\cos$ 值的公式如（4.4）所示。

$$\cos\langle\alpha,\beta\rangle=\frac{x_1x_2+y_1y_2}{\sqrt{x_1^2+y_1^2}\sqrt{x_2^2+y_2^2}} \quad (4.4)$$

其中，两个向量分别为 $\alpha=(x_1,y_1)$ ， $\beta=(x_2,y_2)$ 。

下面将对于不同的向量选择方法进行准确率比较。



图 4.10 系统手势示例

图 4.10 表示系统自带的八种手势以及各自的手势编号，下面将基于这八种手势来测试不同的手部骨架向量选取策略的准确率。

4.4.3 不同的向量选取策略

在本文中，针对手部不同手势的变化，选取了两种不同的向量选取策略。我们对这两种策略进行了详细的测试，旨在通过测试其对图 4.10 所示的各种手势的准确率，找出最佳的选取策略。

在测试中，使用了不同数据集来评估这两种选取策略的准确性和稳定性。我们将这些手势数据集输入到选取策略中，以评估其对不同手势的识别能力。经过多次测试和比较，我们最终找到了相对较优秀的选取策略。这种策略具有很高的准确性和鲁棒性，能够准确地区分不同的手势。在实际应用中，这种选取策略可以大大提高手势识别系统的准确率和稳定性，为用户提供更加便捷和高效的交互方式。

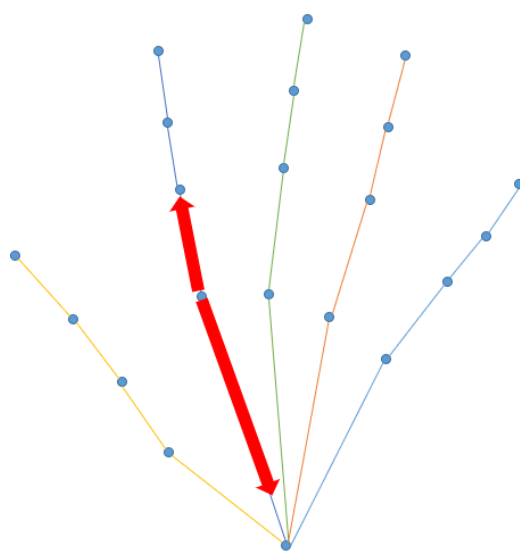


图 4.11 第一种向量选取策略示例

图 4.11 表示第一种向量选取策略，红色箭头表示了二个二维向量。该选取策略的思路是，区分某一个手势的最关键的几个点应该是每个手指与手掌连接的关节点，这五个关节点的弯曲程度从很大程度上决定了手势。所以策略一选择将这五个关节点向两边延伸的两个向量作为计算 $\cos$ 值的向量。该策略对于每只手计算出至少五个 $\cos$ 值存入手势库中。

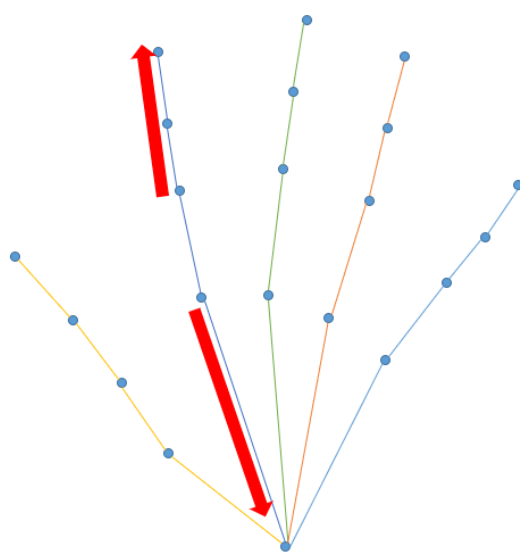


图 4.12 第二种向量选择策略

在图 4.12 中，展示了第一种向量选取策略，其中红色箭头指向了两个二维向量。这种选取策略的思路是，利用手指远端的两个关节和手掌之间的角度来区分不同的手势，而不是依赖于关节的弯曲程度。这种策略具有较高的准确性和稳定性，因为手指的关节弯曲程度会受到许多因素的影响，例如手的姿势和手指的长度等。相比之下，手指远端的两个关节和手掌之间的角度是相对固定的，因此更适合用来区分手势。在该策略中，针对每只手，至少计算出五个 $\cos$ 值，并将其存入手势库中。这些 $\cos$ 值可以用来表示手指远端的两个关节和手掌之间的角度，从而区分不同的手势。通过这种方式，可以更加准确地识别手势，提高手势识别系统的准确性和鲁棒性。

4.4.4 不同策略的比较

使用不同数据集来评估这两种向量选取策略的准确性。我们将这些手势数据集输入到选取策略中，以评估其对不同手势的识别能力。经过多次测试和比较，最终找到了相对较优秀的选取策略。

表 4.1 两种策略结果比较

手势编号	策略一平均准确率（%）	策略二平均准确率（%）
1	87.5	92.5
2	85	95
3	87.5	95
4	90	95
5	97.5	97.5
10	92.5	90
11	90	92.5
8	90	95

从表 4.1 可以看出，策略一的识别准确度总体低于策略二的准确度。可以发现只选择手指与手掌连接的 5 个关节点所得到的结果并没有想象中那么好，原因可能是手部关键点的识别结果与用户的手势有一些差异，间接导致了计算出的 $\cos$ 值的差异，当差异超过阈值的时候，就会识别不正确。

在实际应用过程中，本文的系统将两种方法混合使用，以使手势识别有更好的准确率和鲁棒性。具体来说，可以在五根手指上都应用第二种策略，由于常用的手势更多地用到了大拇指、食指和中指，所以在此基础上对这三根手指再应用第一种策略，可以达到比仅使用第二种向量选取策略更好的准确率。

4.4.5 关键代码

计算两向量的夹角 $\cos$ 值的函数如下所示：

```
def vectors_to_angle(p1, p2, p3, p4):
    x1 = p1[0] - p2[0]
    y1 = p1[1] - p2[1]
    x2 = p3[0] - p4[0]
    y2 = p3[1] - p4[1]
    a = x1 * x2 + y1 * y2
    b = ((x1 ** 2 + y1 ** 2) ** 0.5) * ((x2 ** 2 + y2 ** 2) ** 0.5)
    angle = a / b
    return angle
```

该函数原理就是根据公式（4.4），可以计算出 $p1-p2$ 向量与 $p3-p4$ 两向量的夹角的 $\cos$ 值。

将手部关键点的坐标信息转换成相应的 $\cos$ 值的函数如下：

```
def pose_to_angles(key_point):
    a1 = vectors_to_angle(key_point[0], key_point[1], key_point[2], key_point[4])
    a2 = vectors_to_angle(key_point[0], key_point[5], key_point[6], key_point[8])
    a3 = vectors_to_angle(key_point[0], key_point[9], key_point[10], key_point[12])
    a4 = vectors_to_angle(key_point[0], key_point[13], key_point[14], key_point[16])
    a5 = vectors_to_angle(key_point[0], key_point[17], key_point[18], key_point[20])
    a6 = vectors_to_angle(key_point[1], key_point[2], key_point[2], key_point[3])
    a7 = vectors_to_angle(key_point[5], key_point[6], key_point[6], key_point[7])
    a8 = vectors_to_angle(key_point[9], key_point[10], key_point[10], key_point[11])
    return np.array([a1, a2, a3, a4, a5, a6, a7, a8], dtype=np.float32)
```

其中“key_point”数组存放了 21 个手部关键点，具体编号如图 4.9 所示。本文系统在实际使用中所使用的向量选组策略为上文提到的混合方案，这里仅给出一种示例，还可以有其他的方案。

此外，根据当前收拾去手势库中比对手势以找到最佳手势的函数如下：

```
def __compute_pose(self, key_point):
    angles = pose_to_angles(key_point)
    res = None
    for pose in self.pose_list:
        max = (np.array(pose['angle']) + self.pose_thres >= angles).sum()
        min = (np.array(pose['angle']) - self.pose_thres <= angles).sum()
        if max == min == 8:
            res = pose['name']
    if res is not None:
        return int(res)
    return 0
```

这里本文系统所使用的判断最佳匹配手势的方法为公式（4.3）的方案，该方案计算速度更加快，因为不必和手势库中的所有手势比较完才能确定手势是否相匹配，但是在有多种手势的参数相近的情况下，使用公式（4.2）方案准确率更高。在实际应用过程中可以根据具体需要选择方案。

4.4.6 实现效果

根据一定的向量选取策略，计算出每一个预置手势的相关向量角度信息，存入手势库，即可得到一个 json 文件，文件中存了每个手势以及角度信息，在手势识别前只需要读取该 json 文件并进行比对，按照公式（4.2）或公式（4.3）得到匹配的手势。

```
[{"name": "1", "angle": [-0.012287267483770847, 0.9024254679679871, -0.9852110147476196, 0.017208779230713844, 0.9011473059654236, -0.9941941499710083, 0.535232663154602, 0.9949946999549866, -0.9197759032249451, -0.22223177552223206, 0.9430731534957886, -0.9527571201324463, 0.39579150080680847, 0.9683513641357422, -0.9830557703971863, -0.2964378595352173, 0.995360255241394, -0.9625293016433716, -0.4942343831062317, -0.9128381609916687, -0.9952294230461121, 0.6015809774398804, -0.9845099449157715, -0.95665043592453, -0.38738712668418884, -0.9926443696022034, -0.9875646829605103, 0.454757958650589, -0.8953118920326233, -0.975929319858551, -0.4396376609802246, -0.9992712736129761, -0.9915481209754944, 0.39766013622283936, -0.9595038294792175, -0.9715379476547241, 0.9899019002914429, -0.8967502117156982, -0.9572551250457764, 0.991647481918335, -0.9084070920944214, -0.9595116376876831, -0.9977843165397644, -0.9385979771614075, -0.9539697766304016, 0.9569135308265686, -0.8861396908760071, -0.9196845889091492, 0.9394054412841797, -0.9664754271507263, -0.9999635815620422, 0.9534069299697876, -0.9700537323951721, -0.9976522326469421, 0.9299657344818115, -0.9701478481292725, -0.9938466548919678,
```

图 4.13 手势库数示例

如图 4.13 所示, 手势库中存放着每一种手势, 以及每一种手势对应的角度的 `cos` 信息, 在手势识别时可以进行比对。其中 “name” 为手势编号, “angle” 为存放的角度。

根据以上方法，将图 4.2 中的原图像输入到手势识别的模块中，最终效果如图 4.14 所示。

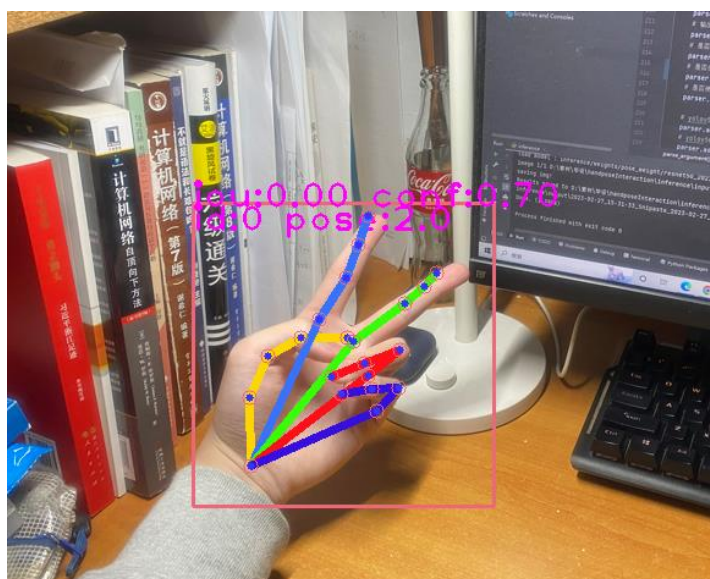


图 4.14 手势识别实现效果

图 4.14 中的“pose:2.0”表明手势识别模块顺利识别出图中手势为手势库中的 2 号手势，由于是对单张图片进行识别，所以 IoU 值为 0。“conf:0.70”表示手部检测的置信度。

4.5 操作逻辑及应用

现在主流操作系统包括 Windows、macOS、Linux、iOS 和 Android。它们都有自己的独特的操作逻辑。例如 Windows 和 macOS 操作系统的操作逻辑非常相似，都使用桌面和任务栏来管理应用程序和文件。用户可以通过鼠标和键盘来控制系统，并使用各种快捷键来提高效率。Linux 操作系统具有高度的定制性和灵活性，用户可以根据自己的需求对系统进行定制和优化。Linux 用户通常使用命令行界面来控制系统，这需要一些学习成本，但也提供了更高的自由度。iOS 和 Android 操作系统是移动设备上的主流操作系统，它们使用触摸屏幕来控制系统。用户可以使用手势来导航和交互，例如滑动、捏合和轻敲。这种交互方式非常直观，但也有一些限制。

AR/VR 技术正在逐渐成为主流，它们的操作逻辑可以借鉴传统操作系统的一些设计。视觉手势交互是人机交互的主要方式之一，为用户提供更大的互动空间，更多的互动自由度和更逼真的互动体验^[19]。例如，桌面和任务栏可以被转换为虚拟桌面和虚拟任务栏，用户可以通过手势来控制它们。快捷键也可以被转换为手势，例如双指捏合来放大和缩小场景，单指轻敲来选择和打开应用程序。此外，AR/VR 技术还可以借鉴操作系统在人机交互方面的一些设计。例如，语音识别和手势识别可以被用来代替键盘和鼠标，让用户更加自然地与系统交互。同时，AR/VR 技术也需要解决一些传统操作系统所没有的问题，例如场景交互、头部追踪等。因此，在借鉴传统操作系统的同时，也需要针对 AR/VR 的特点进行创新和优化。

4.5.1 交互逻辑

好的系统交互逻辑设计直接关系到用户的使用体验。用户使用一款软件时，如果其交互逻辑自然、符合用户使用习惯，那么用户会感到轻松愉悦，相反，如果交互逻辑不自然、难以理解，那么用户使用起来就会感到繁琐和不便。因此，在设计系统交互逻辑时，需要考虑到用户的习惯和感受。

在本文中，系统采用了基于手势识别的交互逻辑设计。不同手势可以被识别为不同的操作，例如“点击”、“长按”等。这种交互方式类似于移动端操作系统，用户可以通过手势来控制系统，而不需要使用键盘和鼠标。这种设计方式不仅更加自然，而且也更

加便捷，用户可以更加轻松地完成操作。此外，系统还采用了基于手势追踪的交互逻辑设计。通过手势追踪，用户可以实现“移动”、“拖动对象”、“下拉”等一系列操作。这种操作方式更加直观和自然，用户可以通过手势来模拟真实操作，使得用户的使用体验更加舒适。

总之，好的系统交互逻辑设计应该是自然的、符合用户的使用习惯的。本文采用基于手势识别和手势追踪的交互逻辑设计，使得用户可以更加自然、便捷地完成操作，这种设计方式更加类似于移动端的操作系统，使得用户使用起来更加得心应手。

4.5.2 应用开发

本文系统利用上文所提到的操作逻辑，开发了图像内容识别、虚拟键盘、虚拟钢琴等应用。

具体地，图像内容识别应用允许用户选择画面中的某一区域，对该区域内的物体进行识别，并将结果输出。虚拟键盘应用实现了在画面中显示虚拟的键盘，用户可以通过点击屏幕上的虚拟按键来输入字符。虚拟钢琴应用则实现了在画面中显示虚拟的钢琴，并通过多线程来使得系统发出声音，让用户可以在屏幕上弹奏出曲子。以上三个应用均通过菜单界面进入，并在应用内使用上文所提到的操作逻辑。

4.5.3 菜单界面

菜单界面是系统中非常重要的界面之一，它可以实现应用切换和一些操作的功能。在主界面上，用户可以通过“下拉”动作进入菜单界面，然后选择需要使用的应用。菜单界面通常包括应用图标、名称、描述以及一些操作按钮等元素，这些元素的设计直接影响用户的使用体验。一个好的菜单界面应该能够让用户快速找到需要使用的应用和功能，并且操作简单方便，提高用户的使用体验。

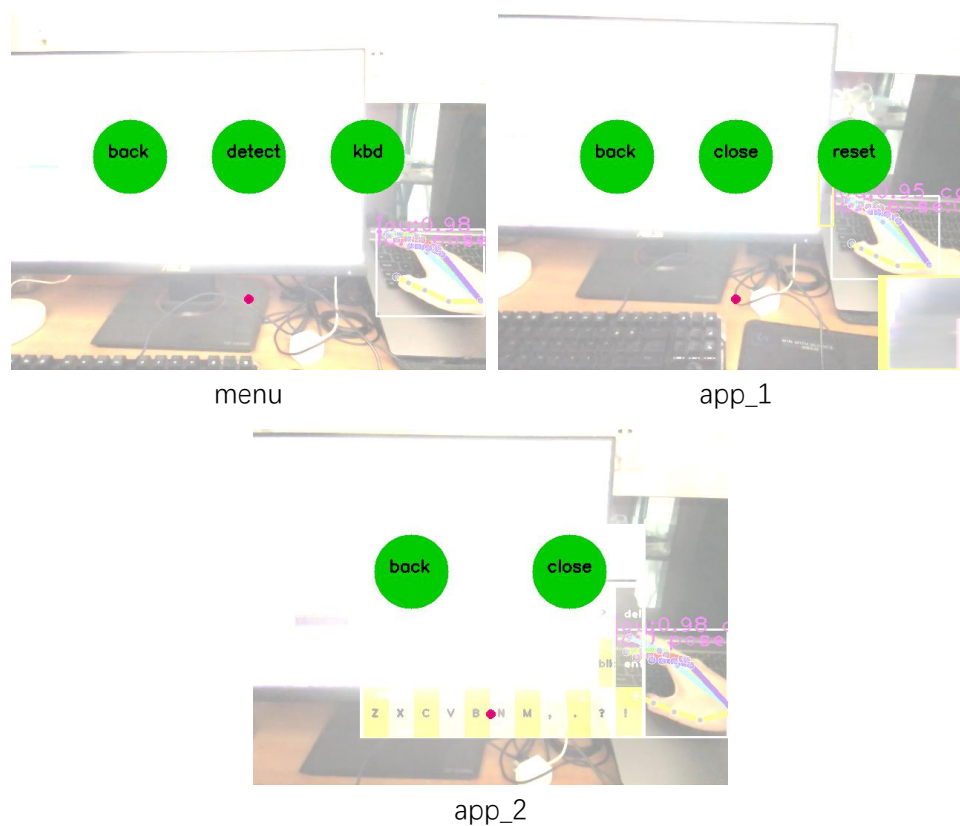


图 4.15 菜单界面

图 4.15 中即为在不同应用下的菜单界面，绿色圆圈是交互按钮，用户通过系统手势操作紫红色光标，将光标移动到按钮内松开即触为点击该按钮。“back”按钮作用为关闭菜单，返回到当前应用。在“menu”界面中的“detect”按钮作用是进入图像内容识别应用，“kbd”按钮作用是进入虚拟键盘应用。“app_1”界面是在图像内容识别应用中时的菜单界面，其中“close”按钮作用是关闭当前应用，“reset”按钮作用是重置图像内容识别应用的识别框位置、或者重新开始识别。“app_2”界面是在虚拟键盘应用中时的菜单界面，其中“close”按钮作用是关闭当前应用。

例如，在没有开启应用的情况下的菜单界面的实现代码如下所示：

```
if self.app_start == 0:
    c1_x, c1_y = int(width / 4), int(height / 5) * 2
    c2_x, c2_y = int(width / 4) * 2, int(height / 5) * 2
    c3_x, c3_y = int(width / 4) * 3, int(height / 5) * 2
    c_radius = 50
    cv2.circle(im0, (c1_x, c1_y), c_radius, (0, 204, 0), -1)
    cv2.putText(im0, 'back', (c1_x - 30, c1_y), cv2.FONT_HERSHEY_SIMPLEX, 0.75,
(0, 0, 0), 2)
    cv2.circle(im0, (c2_x, c2_y), c_radius, (0, 204, 0), -1)
    cv2.putText(im0, 'detect', (c2_x - 30, c2_y), cv2.FONT_HERSHEY_SIMPLEX, 0.75,
(0, 0, 0), 2)
    cv2.circle(im0, (c3_x, c3_y), c_radius, (0, 204, 0), -1)
    cv2.putText(im0, 'kbd', (c3_x - 20, c3_y), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0,
0, 0), 2)
    cv2.circle(im0, (self.menu_mouse[0], self.menu_mouse[1]), 5, (153, 0, 204), -1)
    cv2.circle(im0, (self.menu_mouse[0], self.menu_mouse[1]), 5, (102, 0, 255), 2)
```

其中隐去了对按钮的点击判断的代码。

4.5.4 图像内容识别应用

图像内容识别应用是一种智能化的应用程序，可以为用户提供准确的图像识别功能。用户可以选择一片区域，并对该区域内的图像内容进行识别。在本文系统中，我们使用了 YOLOv5 目标检测算法来实现图像内容识别功能，该算法可以检测出二十多个大类的物品，如人、车、飞机、动物、器具等等。

这种图像内容识别应用具有广泛的应用场景，例如在教学领域，可以使用该应用来识别书本上的图画，为学生提供更为丰富和准确的教学资源。用户只需要选择书本上的某个图画，然后将该图画传入本文系统，系统即可对该图画进行自动识别，并提供相应的识别结果。这种识别功能可以大大提高教学效率和学生的学习体验。

除此之外，图像内容识别应用还可以在其他领域中得到应用，例如在安防监控领域中，可以使用该应用来识别监控画面中的人和车等物品，以及监控画面中的异常情况。这种应用可以帮助用户更快速、准确地发现和解决问题，提高工作效率和安全性。

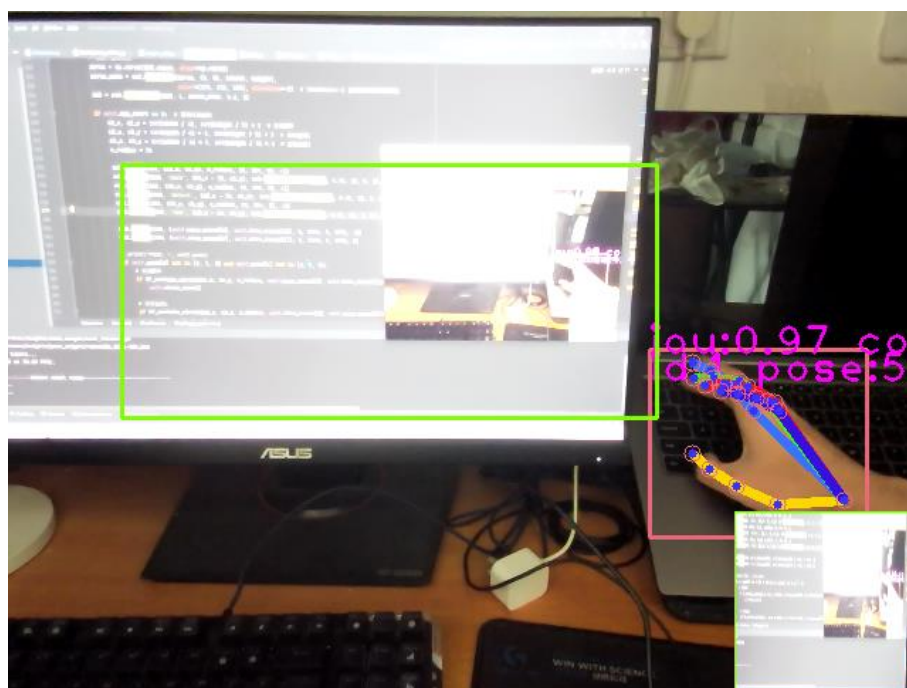


图 4.16 划定选框

总之，图像内容识别应用可以为用户提供准确的图像识别功能，具有广泛的应用场景。本文系统使用了 YOLOv5 目标检测算法来实现图像内容识别功能，可以检测出二十多个大类的物品，用户可以将其应用在教学、安防监控等领域中，为用户提供更为丰富和准确的应用体验。

应用 1 的界面所涉及的关键代码如下所示：

```
if self.app_dict["left_top"] is not None and self.app_dict["right_bottom"] is not None and
self.app_start == 1:
    cv2.rectangle(im0, (self.app_dict["left_top"][0], self.app_dict["left_top"][1]),
(self.app_dict["right_bottom"][0], self.app_dict["right_bottom"][1]), (0, 255, 127), 2)
    if self.app_dict["result"] is not None:
        cv2.putText(im0, self.app_dict["result"]["class"], (self.app_dict["left_top"][0],
self.app_dict["left_top"][1]), cv2.FONT_HERSHEY_PLAIN, 2, [255, 0, 255], 2)
        img_cut = im0[int(self.app_dict["left_top"][1]):int(self.app_dict["right_bottom"][1]),
int(self.app_dict["left_top"][0]):int(self.app_dict["right_bottom"][0])]
        h_ori, w_ori = im0.shape[:2]
        h, w = 128, 128
        if int(self.app_dict["right_bottom"][1]) - int(self.app_dict["left_top"][1]) > 1 and
int(self.app_dict["right_bottom"][0]) - int(self.app_dict["left_top"][0]) > 1:
            img_cut = cv2.resize(img_cut, (w, h))
            im0[(h_ori - h):h_ori, (w_ori - w):w_ori] = img_cut
```

其中隐去了多线程实现目标检测的代码。

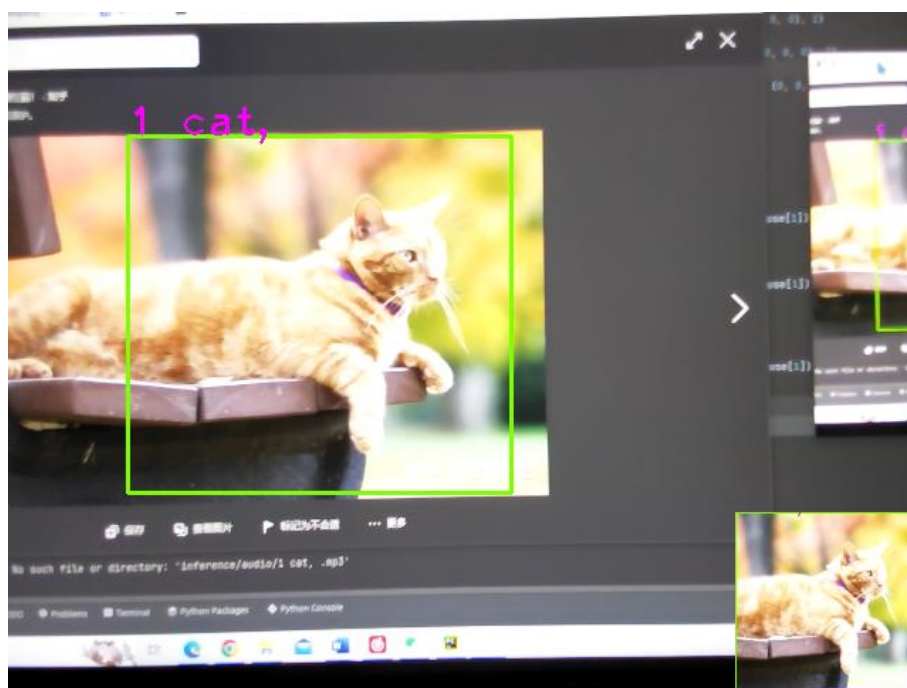


图 4.17 识别结果

如图 4.16 及图 4.17 所示，用户可以通过自行框定画面中想要识别的区域来使用本文系统的目标检测功能。用户可以通过系统手势划动或点击来框定目标区域，框定完成后，可以使用本文系统中的目标检测模型来对所选区域进行识别。

在本文系统中，触发目标检测的条件是用户做出图 4.10 中的 11 号手势，也就是在选择区域后，用户需要做出相应的手势来触发检测模型。这种设计方式既方便了用户，也增加了系统的智能化和交互性。

框内图像的识别结果将会显示在画面中，“1 cat”就是框定区域图像的识别结果，它表示在用户所选区域中识别出了一只猫。这种图像识别功能可以为用户提供更为准确的识别结果，方便用户进行相应的操作。

总之，本文系统中的目标检测功能可以通过用户框定区域和做出特定手势的方式来触发，同时，该功能能够准确识别出用户所选区域的图像内容，并为用户提供相应的识别结果，从而为用户提供更加智能化和便利的操作体验。

4.5.5 虚拟键盘应用

虚拟键盘应用可以让用户与虚拟键盘进行交互，用户可以在键盘上打字。虚拟键盘采用了经典的键盘布局，用户使用起来可以快速上手。



图 4.18 虚拟键盘应用

如图 4.18 所示，虚拟键盘中包含 26 个英文字母，“,”、“.”、“?”、“!” 四个标点，以及退格、回车和空格共 33 个按键。用户还可以调整虚拟键盘的位置，只需要做出图 4.10 中的 10 号手势即可对虚拟键盘进行拖动操作。处于对在某些情况下的安全性以及应用使用的舒适性考虑，将虚拟键盘设置成了半透明的样式，这样用户在使用的时候也不会遮挡画面。

键盘应用界面的代码如下所示：

```
if self.app_start == 2:
    kb_h, kb_w = int(height * 0.6), int(width * 0.6)
    if self.kb_pos[0] == 0 and self.kb_pos[1] == 0:
        self.kb_pos = np.array([int(height * 0.3), int(width / 2 - kb_w / 2)])
    b_w = int(kb_w / 11)
    b_h = int(kb_h * 0.7 / 3)
    for i, o in enumerate(order):
        if o[2] == 10:
            if self.pose_last[i] != 10:
                self.last_pos = np.array([[-1, -1, -1], [-1, -1, -1]])
            if self.last_pos[i][0] != -1 and self.last_pos[i][1] != -1:
                self.delta_x = o[0] - self.last_pos[i][0]
                self.delta_y = o[1] - self.last_pos[i][1]
            self.last_pos[i] = o
            self.kb_pos[0] += self.delta_x
```

虚拟键盘的位置可以由用户自行移动，其中隐去了更新每个按键的位置等代码。

4.5.6 虚拟钢琴应用

虚拟钢琴应用在画面中呈现了一个虚拟的钢琴键盘，界面类似于虚拟键盘应用，以简化真实钢琴的复杂性，它只包含了七个白键和五个黑键。在本系统中，第一个白键表示音调 do，之后的每个白键音调递增一个音符，即 do、re、mi、fa、so、la、si。而黑键被定义为比相邻左侧白键高半个音符。这个设计充分考虑到了钢琴的音乐理论和实际演奏的需要，同时也能够让用户更轻松地上手操作虚拟钢琴。

在虚拟钢琴应用中，为了实现发出真实的声音效果，系统使用了多线程。在用户按下钢琴键时，系统会通过判断按下的键位来确定要发出的音调，并从声音存储文件中找

到响应的音调文件。然后，系统会开启一个新的线程来播放对应频率的声音。由于多线程可以实现同时播放多个声音，因此可以使得多个键位同时被按下时不会出现声音的错乱或延迟。同时，系统还使用了一些技巧来优化声音的质量和延迟，如使用较小的缓冲区来减少延迟，并对声音进行加速和减速来适应不同的键位。最终，用户可以通过按下虚拟钢琴的按键来发出真实的钢琴声音。

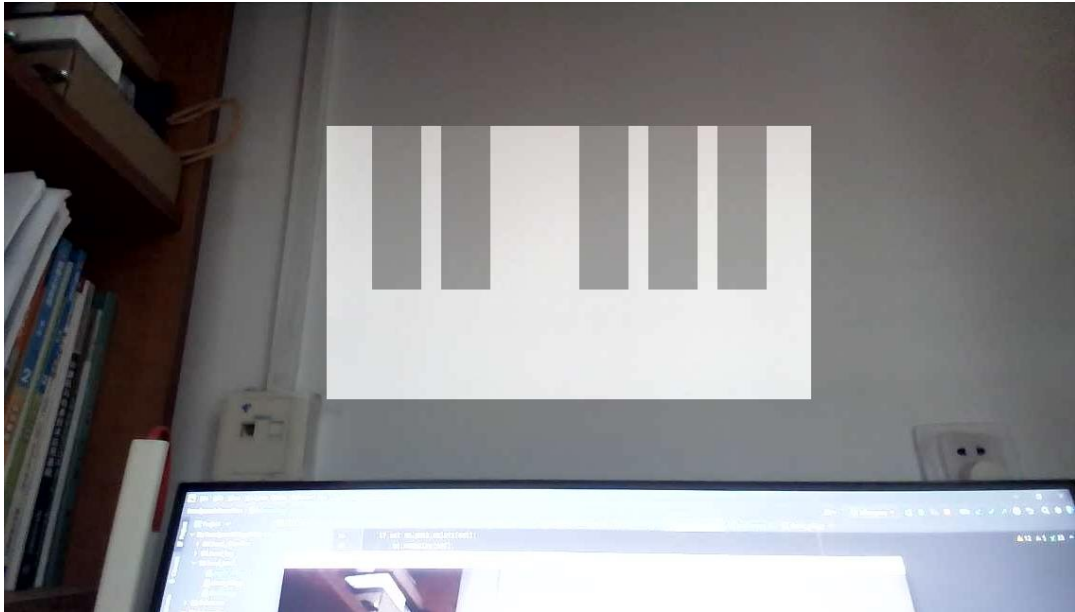


图 4.19 虚拟钢琴应用

如图 4.19 所示，该应用画面中有虚拟钢琴对象，包含了七个白键和五个黑键，用户可以对其进行交互从而实现弹奏。

5 结论与展望

本增强现实交互系统基于手部关键点检测模块，对各个功能模块进行了详细的开发。本系统采用 Python 编写，在 PyCharm Community 平台开发，本系统所使用的计算机搭载 MX250 显卡，系统采用 NVIDIA CUDA 来使用 GPU 加速计算，摄像头使用了 1080P 高清摄像头。在手部检测功能方面，本文系统使用 YOLOv5 目标检测算法。在关键点检测方面，本文系统采用基于深度学习的手部关键点检测的模型。对于手部追踪方面，本文系统基于 IoU 的计算进行手部轨迹的追踪。对于手势识别方面，本文系统采用了基于手部骨架的手势识别，充分利用了手部关键点信息。交互逻辑方面，系统中不同手势可以被识别为不同的操作，例如“点击”、“长按”等。利用这些交互逻辑，本文系统开发了图像内容识别、虚拟键盘、虚拟钢琴等应用。

本文系统还存在一些问题。第一，本文的手势识别采用了基于手部骨架的手势识别，利用检测出的关键点得到手部姿态的信息，包括特定关键点组成的向量之间的角度，以此进行手势的识别。虽然该方法所需要的图片数量很少，但是所适用的手势的特点是各个关节角度差异明显，且角度变化范围较小，使用该方法进行手势识别的精度依赖于关键点检测的精度，对于一些复杂的手势或者与现有手势特征较为相近的手势，识别准确率较低。改进方法可以是使用深度学习的方法来进行手势识别。第二，本文系统的优化还需要改进，由于设备性能的限制，在我的设备上处理一帧图片的时间需要一百毫秒左右，也就是实时画面每秒仅有大约 10 帧，改进方法可以是优化系统的代码或者使用性能更好的设备。

参考文献

- [1] Caputo A, Giachetti A, Soso S, et al. SHREC 2021: Skeleton-based hand gesture recognition in the wild[J]. COMPUTERS AND GRAPHICS, 2021, 99:201-211.
- [2] 吴文斌. 基于深度学习的手部关键点检测及其移动端应用[D]. 华南理工大学, 2019.DOI:10.27151/d.cnki.ghnlu.2019.001554.
- [3] 张圆圆. 基于视觉的手势识别技术及其应用研究[J]. 计算技术与自动化, 2015, 34(1):131-135
- [4] 马哲. 驾驶员手部检测和驾驶行为分析研究[D]. 济南大学, 2021.DOI:10.27166/d.cnki.gsdcc.2021.000084.
- [5] Facebook. Oculus Quest Hand Tracking. <https://www.oculus.com/blog/oculus-connect-6-introducing-hand-tracking-on-oculus-quest-facebook-horizon-and-more/>.
- [6] 陈帅, 袁宇浩. 改进 Yolov5 的手语字母识别算法研究[J/OL]. 小型微型计算机系统:1-9[2023-04-14]. <http://kns.cnki.net/kcms/detail/21.1106.TP.20220215.1042.017.html>
- [7] 王森宝, 杨晋骁, 王子昂, 李世尧, 秦娟, 石艳梅. 基于手部关键点检测的手势识别研究[J]. 电脑与电信, 2022(5):29-3347
- [8] Zhang F, Bazarevsky V, Vakunov A, et al. (2020). Mediapipe hands: On-device real-time hand tracking. arXiv preprint arXiv:2006.10214. <https://arxiv.org/abs/2006.10214>
- [9] Francisco Gomez-Donoso, Sergio Orts-Escolano, and Miguel Cazorla. "Large-scale Multiview 3D Hand Pose Dataset". ArXiv e-prints 1707.03742, July 2017.
- [10] M. A. Kassab, M. Ahmed, A. Maher, and B. Zhang. Real-time human-UAV interaction: New dataset and two novel gesture-based interacting systems [J]. IEEE Access, vol. 8, pp. 195030–195045.
- [11] S. Jacob, V. G. Menon, F. Al-Turjman, P. G. Vinoj, and L. Mostarda. Artificial muscle intelligence system with deep learning for post-stroke assistance and rehabilitation [J]. IEEE Access, vol. 7, pp. 133463–133473, Sep. 2019.
- [12] G. Wang, Q. Li, L. Wang, Y. Zhang, and Z. Liu. Elderly fall detection with an accelerometer using lightweight neural networks [J]. Electronics, vol. 8, no. 11, p. 1354, Nov. 2019.
- [13] M. Al-Hammadi, et al. Deep learning-based approach for sign language gesture recognition with efficient hand gesture representation [J]. IEEE Access, vol. 8, pp. 192527–

192542, Oct. 2020.

[14] C. Lee, J. Kim, S. Cho, J. Kim, J. Yoo, and S. Kwon. Development of real-time hand gesture recognition for tabletop holographic display interaction using azure Kinect [J]. *Sensors* (Basel), vol. 20, no. 16, p. 4566, Aug. 2020.

[15] X. Yu, L. Jiang, and L. Wang. Virtual reality gesture recognition based on depth information [J]. *SID Int. Symp. Dig. Tech. Paper*, Copenhagen, Denmark, 2020, pp. 196–200.

[16] 吴江琴, 高文. 基于 ANN/HMM 的手语识别方法[J]. *计算机科学*, 1999(10).

[17] J. Davis, M. Shah. Visual gesture recognition [C]//*Proceeding on Vi-sion, Image Signal Processing*, 1994:321-332.

[18] 翟艺杰. 基于深度学习的高效手部检测和手势识别方法研究[D]. 桂林电子科技大学, 2021. DOI:10.27049/d.cnki.ggldc.2021.000911.

[19] 蔡小龙. 增强现实中的手势交互系统研究与设计[D]. 重庆大学, 2018.

致谢

四年的大学时光转瞬即逝，在此，我要向所有在我大学本科时光中，给予我力量与帮助，为我照亮人生之路的人致以最衷心的感谢！首先我要感谢我的家人们，一直在背后默默地支持我，尊重我的每一个选择，并给予我正确的人生指导，教我如何做人。他们是我坚强的后盾，也是我奋力拼搏的动力，我定将继续努力完善自我，提升自我，变得更加优秀。

然后，我要感谢的人莫过于我的导师陈老师，他给了我很多机会与平台来提升自我、展现自我。我认为在这四年里做的正确的选择之一就是在大一时向陈老师申请加入了 ACM 集训队，虽然我的成绩并没有很突出，但是在集训队期间我的编程能力以及思考解决问题的能力有了非常大的提升，也让我养成了去静下心来专心学习和理解新的事物的习惯，这一点在我备考研究生考试的过程中让我受益良多。

在集训队中我遇到了许多志同道合的朋友，我也要感谢他们。回想起在赛前训练的时光，和队友们一起解出一道又一道题目，虽然在这过程中有时候也有遗憾，但是我仍旧十分享受这段时间，每每回想起都让我感觉热血沸腾。我和章鑫鹏同学做了两年队友，我们又共同备战研究生考试，最终都被理想学校录取，将来我们又将一起度过三年的研究生生活。

我还要感谢教导过我的老师们、我的学长学姐们、我的舍友们、19 计算机 1 班的同学们。这段时间一起的回忆将永存于心。毕业不代表散场，相信我们的缘分还在继续。愿我们能不忘初心，进德修业，与时偕行！